

Lección 29: Performance Percibida

Hacer que tu App se Sienta Rápida

Agenda

- Performance Percibida vs Real
 - Psicología de la Espera
 - Pantallas Esqueleto (Skeleton Screens)
 - UI Optimista (Optimistic UI)
 - Carga Progresiva (Progressive Loading)
 - Estrategias de Precarga
 - Feedback Instantáneo
 - Indicadores de Carga
 - Mejores Prácticas
-

Performance Percibida vs Real

Performance real (objetivo):

Medido en milisegundos

Lo que realmente sucede:

- Tiempo de respuesta API: 500ms
- Tiempo de renderizado: 200ms
- Tiempo total: 700ms

Herramientas: Lighthouse, Network tab

Performance percibida (subjetivo):

Qué tan rápido se SIENTE para los usuarios

Lo que el usuario experimenta:

- ¿Se siente instantáneo?
- ¿Se siente responsivo?
- ¿Se siente suave?

No puede medirse directamente

Insight clave: Los usuarios recuerdan cómo se sintió, no los milisegundos reales

Psicología de la Espera

Percepción humana del tiempo:

< 100ms: Instantáneo

- Se siente como manipulación directa
- No necesita feedback

100–300ms: Retraso leve

- Usuario nota que algo sucedió
- Feedback simple (cambio de estado)

300ms–1s: Retraso notable

- Necesita indicador de carga
- Usuario permanece comprometido

1–3s: Espera significativa

- Necesita indicador de progreso
- Usuario puede cambiar de contexto

> 3s: Espera larga

- Usuario probablemente abandonará
- Necesita distracción o aseguración

Objetivo: Mantener usuarios en rango < 1s tanto como sea posible

El Costo de la Lentitud

Comportamiento del usuario:

0-1s: Feliz, comprometido
1-2s: Ligeramente impaciente
2-3s: Frustrado
3-5s: Muy frustrado
> 5s: Abandona la tarea

Abandono:

- 1s → 3s: +32% tasa de rebote
- 1s → 5s: +90% tasa de rebote
- 1s → 10s: +123% tasa de rebote

Impacto en ingresos:

Amazon: +100ms = -1% ventas
Google: +500ms = -20% tráfico
Walmart: -1s tiempo de carga = +2% conversiones

No siempre podemos hacer las cosas más rápidas,
pero podemos hacer que se SIENTAN más rápidas

Pantallas Esqueleto (Skeleton Screens) (1/2)

¿Qué son?:

Placeholder mostrando estructura del contenido mientras carga
Usuarios ven layout inmediatamente → se siente instantáneo
Contenido se llena mientras carga → se siente progresivo

✗ Pantalla en blanco:

```
{  
  isLoading ? <div>Cargando...</div> : <ProductList products={products} />  
}
```

El usuario ve: Pantalla en blanco por 1-2 segundos → Contenido aparece

Pantallas Esqueleto (Skeleton Screens) (2/2)

✓ Pantalla esqueleto:

```
{
  isLoading ? (
    <ProductListSkeleton /> // Muestra estructura inmediatamente
  ) : (
    <ProductList products={products} />
  )
}
```

El usuario ve: Estructura instantáneamente → Contenido se llena

Se siente 2x más rápido ¡aunque el tiempo real es el mismo!

Ejemplo de Pantalla Esqueleto

```
const ProductSkeleton = () => (
  <div className="product-card animate-pulse">
    {/* Placeholder de imagen */}
    <div className="w-full h-48 bg-gray-200 rounded" />

    {/* Placeholder de título */}
    <div className="mt-4 h-4 bg-gray-200 rounded w-3/4" />

    {/* Placeholder de precio */}
    <div className="mt-2 h-4 bg-gray-200 rounded w-1/4" />

    {/* Placeholder de botón */}
    <div className="mt-4 h-10 bg-gray-200 rounded" />
  </div>
)

const ProductListSkeleton = () => (
  <div className="grid grid-cols-3 gap-4">
    {[...Array(6)].map((_, i) => (
      <ProductSkeleton key={i} />
    ))}
  </div>
)
```

```
// Uso
function ProductsPage() {
  const { data: products, isLoading } = useProducts()

  return (
    <div>
      <h1>Productos</h1>
      {isLoading ? <ProductListSkeleton /> : <ProductList products={product
    </div>
  )
}
```

UI Optimista (Optimistic UI)

Flujo tradicional (pesimista):

1. Usuario hace click "Agregar al Carrito"
2. Esperar respuesta del servidor
3. Mostrar spinner de carga (500ms)
4. Actualizar UI con respuesta
5. Mostrar mensaje de éxito

Usuario espera: 500ms

Flujo optimista:

1. Usuario hace click "Agregar al Carrito"
2. Actualizar UI inmediatamente (agregar ítem al carrito)
3. Enviar request al servidor en segundo plano
4. Si éxito: ¡Listo! (ya actualizado)
5. Si error: Revertir + mostrar error

Usuario espera: 0ms (se siente instantáneo!)

Ejemplo de UI Optimista

```
const addToCart = async (product: Product) => {
  // 1. Actualización optimista (instantánea)
```

```

setCart(prev => [...prev, product])
setCartCount(prev => prev + 1)

// 2. Mostrar feedback instantáneo
toast.success(`✓ ${product.name} agregado al carrito!`)

try {
  // 3. Guardar en servidor (segundo plano)
  await api.post('/cart', { productId: product.id })
} catch (error) {
  // 4. Revertir en caso de error
  setCart(prev => prev.filter(p => p.id !== product.id))
  setCartCount(prev => prev - 1)

  // 5. Mostrar error
  toast.error('No se pudo agregar al carrito. Intenta de nuevo.')
}
}

// Usuario ve:
// - Actualización instantánea del carrito (optimista)
// - Toast de éxito instantáneo
// - Si la red es lenta, no importa (ya actualizado)
// - Si hay error, revierte suavemente

```

Se siente instantáneo ¡incluso en redes lentas!

Cuándo Usar UI Optimista (1/2)

 **Bueno para:**

- Agregar al carrito
- Dar like/favoritos
- Actualizar cantidad
- Cambiar configuraciones
- Enviar mensajes
- Eliminar ítems

Requerimientos:

- Fácil de revertir
 - Baja tasa de fallo
 - Operaciones no críticas
-

Cuándo Usar UI Optimista (2/2)

✗ Evitar para:

- Procesamiento de pagos
- Creación de cuentas
- Cambios de contraseña
- Eliminación de datos (si es irreversible)
- Operaciones críticas

Requerimientos:

- Necesita confirmación del servidor
- Alto riesgo
- No se puede revertir fácilmente

Carga Progresiva (Progressive Loading)

Cargar contenido crítico primero, menos importante después:

```
function ProductPage({ productId }) {  
  // 1. Cargar datos críticos inmediatamente  
  const { data: product } = useQuery(['product', productId], {  
    staleTime: 5000,  
  })  
  
  // 2. Cargar datos secundarios después (lazy)  
  const { data: reviews } = useQuery(['reviews', productId], {  
    enabled: !!product, // Solo después de que cargue el producto  
    staleTime: 60000,  
  })  
  
  // 3. Cargar datos terciarios al final (bajo demanda)  
  const [showRecommendations, setShowRecommendations] = useState(false)  
  const { data: recommendations } = useQuery(['recommendations', productId], {  
    enabled: showRecommendations,  
  })  
  
  return (  
    <div>  
      { /* Crítico: Mostrar inmediatamente */ }  
      <ProductDetails product={product} />  
    </div>  
  )  
}
```

```
    { /* Secundario: Mostrar cuando esté listo */ }
    { reviews ? <Reviews reviews={reviews} /> : <ReviewsSkeleton /> }

    { /* Terciario: Cargar bajo demanda */ }
    <button onClick={() => setShowRecommendations(true)}>Mostrar Producto
    {showRecommendations && <Recommendations items={recommendations} />}
  </div>
)
}
```

El usuario ve contenido crítico rápido, resto carga progresivamente

Estrategias de Carga de Imágenes (1/2)

✗ Sin optimización:

```

// Carga imagen completa, bloquea render, muestra espacio en blanco
```

✓ Placeholder borroso:

```
<div className="relative">
  { /* Versión pequeña borrosa carga primero (2KB) */ }
  

  { /* Imagen completa carga en segundo plano */ }
   setLoaded(true)} />
</div>
```

Estrategias de Carga de Imágenes (2/2)

✓ Lazy loading:

```

```



```
// 0 para más control:
import { LazyLoadImage } from 'react-lazy-load-image-component'
<LazyLoadImage src="/product.jpg" placeholderSrc="/product-tiny.jpg" effect="blur" />
```

Estrategias de Precarga (1/2)

Precargar al pasar mouse (especulativo):

```
const ProductCard = ({ product }) => {
  const prefetch = usePrefetch()

  return (
    <Link
      to={` /product/${product.id}`}
      onMouseEnter={() => {
        // Usuario sobre el elemento → probablemente hará click
        // Precargar detalles del producto
        prefetch(['product', product.id])
      }}
    >
      <img src={product.image} />
      <h3>{product.name}</h3>
    </Link>
  )
}

// Cuando el usuario hace click:
// Datos ya cargados → ¡Instantáneo!
```

Estrategias de Precarga (2/2)

Precargar siguiente paso:

```
// En página de carrito, precargar datos de checkout
useEffect(() => {
  if (cart.length > 0) {
    // Usuario probablemente hará checkout → precargar
    prefetchCheckoutData()
  }
})
```

```
}, [cart])
```

```
// Cuando usuario hace click "Checkout":  
// Ya cargado → ¡Se siente instantáneo!
```

Feedback Instantáneo

Cada interacción necesita feedback inmediato:

```
// ❌ Sin feedback  
<button onClick={handleClick}>  
  Agregar al Carrito  
</button>  
  
// ✅ Cambio de estado inmediato  
<button  
  onClick={handleClick}  
  className={isAdding ? 'opacity-50' : ''}  
  disabled={isAdding}  
>  
  {isAdding ? (  
    <>  
      <Spinner /> Agregando...  
    </>  
  ) : (  
    'Agregar al Carrito'  
  )}  
</button>  
  
// ✅ Aún mejor: Actualización optimista  
<button onClick={() => {  
  // Actualización de UI instantánea  
  setCartCount(prev => prev + 1)  
  showToast('¡Agregado al carrito!')  
  
  // Guardar en segundo plano  
  addToCartAsync(product)  
}}>  
  Agregar al Carrito  
</button>
```

Regla: < 100ms de respuesta a cada acción del usuario

Indicadores de Carga (1/2)

Cuándo mostrar:

< 300ms: No indicator needed (se siente instantáneo)
300ms-1s: Simple spinner
1-3s: Progress indicator
> 3s: Progress + time estimate

Indicadores de Carga (2/2)

Tipos:

Spinner (duración desconocida):

```
<Spinner /> Procesando...
```

Barra de progreso (pasos conocidos):

```
<ProgressBar value={step} max={3} />  
Paso {step} de 3
```

Skeleton (carga de contenido):

```
<ProductListSkeleton />
```

Porcentaje (subida de archivo):

```
<ProgressBar value={uploadProgress} max={100} />  
{uploadProgress}% subido
```

Animación para Velocidad Percibida (1/2)

Las transiciones suaves se sienten más rápidas:

```
/* ❌ Instant (se siente brusco) */
.product {
  display: none;
}
.product.show {
  display: block;
}

/* ✅ Smooth (se siente pulido) */
.product {
  opacity: 0;
  transform: translateY(20px);
  transition: all 200ms ease-out;
}
.product.show {
  opacity: 1;
  transform: translateY(0);
}
```

Animación para Velocidad Percibida (2/2)

Animaciones escalonadas (se siente más natural):

```
{
  products.map((product, index) => (
    <div
      key={product.id}
      className="product fade-in"
      style={{
        animationDelay: `${index * 50}ms`, // Escalonar por 50ms
      }}
    >
      <ProductCard product={product} />
    </div>
  ))
}
```

Percepción del usuario: “¡Wow, esto se siente suave y rápido!”

Estrategias de Caché

Cachear agresivamente para sentirse instantáneo:

```
// React Query caching
const { data: products } = useQuery(['products'], fetchProducts, {
  staleTime: 5 * 60 * 1000, // 5 minutos fresco
  cacheTime: 30 * 60 * 1000, // 30 minutos en caché

  // Mostrar datos en caché inmediatamente
  initialData: getCacheProducts(),
  initialDataUpdatedAt: getCacheTimestamp(),
})

// Usuario navega de vuelta a página de productos:
// → Instantáneo (desde caché)
// → Datos frescos cargan en segundo plano si están viejos

// LocalStorage para carrito
useEffect(() => {
  // Carga instantánea desde localStorage
  const cached = localStorage.getItem('cart')
  if (cached) {
    setCart(JSON.parse(cached))
  }

  // Sincronizar con servidor en segundo plano
  syncCartWithServer()
}, [])
```

Se siente instantáneo ¡incluso en redes lentas!

Service Workers para Uso Offline

Cachear assets para carga instantánea:

```
// service-worker.js
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open('v1').then(cache => {
      return cache.addAll(['/', '/index.html', '/styles.css', '/app.js', '/
    })
  )
})
```

```
self.addEventListener('fetch', event => {
  event.respondWith(
    // Intentar caché primero (instantáneo)
    caches.match(event.request).then(response => {
      // Retornar versión en caché o hacer fetch
      return response || fetch(event.request)
    })
  )
})
```

Resultado: ¡App carga instantáneamente incluso offline!

Midiendo Performance Percibida

Métricas:

Time to First Byte (TTFB):

→ Qué tan rápido responde el servidor

First Contentful Paint (FCP):

→ Cuándo el usuario ve algo

Largest Contentful Paint (LCP):

→ Cuándo el contenido principal es visible

Time to Interactive (TTI):

→ Cuándo el usuario puede interactuar

First Input Delay (FID):

→ Qué tan rápido responden las interacciones

Objetivo:

FCP: < 1.8s

LCP: < 2.5s

TTI: < 3.8s

FID: < 100ms

Herramientas: Lighthouse, WebPageTest, Sentry Performance

Mejores Prácticas (1/2)

1. Mostrar algo instantáneamente:

- ✓ Pantallas esqueleto
- ✓ Contenido en caché
- ✓ Estructura de layout
- ✗ Pantalla en blanco

2. Proporcionar feedback inmediato:

- ✓ Cambios de estado de botones
- ✓ Actualizaciones optimistas
- ✓ Transiciones instantáneas
- ✗ Sin respuesta a clicks

Mejores Prácticas (2/2)

3. Cargar progresivamente:

- ✓ Contenido crítico primero
- ✓ Contenido secundario después
- ✓ Contenido terciario bajo demanda
- ✗ Esperar por todo

4. Cachear agresivamente:

- ✓ Cachear respuestas API
- ✓ Cachear imágenes
- ✓ Cachear rutas
- ✗ Hacer fetch cada vez

Mejores Prácticas (cont.) (1/2)

5. Usar UI Optimista:

- ✓ Agregar al carrito instantáneamente
- ✓ Actualizar cantidad instantáneamente
- ✓ Revertir en error
- ✗ Esperar al servidor

6. Precargar inteligentemente:

- ✓ Precargar al pasar mouse
- ✓ Precargar siguiente paso
- ✓ Precargar imágenes
- ✗ Precargar todo (desperdicia ancho de banda)

Mejores Prácticas (cont.) (2/2)

7. Animaciones suaves:

- ✓ Fade in/out
- ✓ Transiciones de deslizamiento
- ✓ Carga escalonada
- ✗ Cambios bruscos instantáneos

8. Estados de carga claros:

- ✓ Mensajes contextuales
- ✓ Indicadores de progreso
- ✓ Estimaciones de tiempo
- ✗ "Cargando..." genérico

Checklist de Performance Percibida (1/2)

Carga Inicial:

- ☑ Mostrar esqueleto mientras carga
- ☑ Cachear assets estáticos
- ☑ Lazy load de imágenes

- ☒ Code splitting

Interacciones:

- ☒ < 100ms feedback en cada click
- ☒ Actualizaciones optimistas
- ☒ Transiciones suaves
- ☒ Sin estados en blanco

Checklist de Performance Percibida (2/2)

Navegación:

- ☒ Precargar al pasar mouse
- ☒ Cachear rutas
- ☒ Transiciones de página instantáneas
- ☒ Preservar posición de scroll

Carga:

- ☒ Mostrar progreso para operaciones largas
- ☒ Proporcionar estimaciones de tiempo
- ☒ Mantener al usuario informado
- ☒ Permitir cancelación

Ejercicio 1: Loading States con Feedback Instantáneo

Prompt:

Actúa como un frontend developer implementando estados de carga con feedback

CONTEXT0: Perceived Performance = cómo se SIENTE la app (NO cuánto tarda re Feedback instantáneo < 100ms = se siente manipulación directa. Button state loading, success, error (estados visuales claros). Disabled state: previene mientras procesa. Visual feedback: spinner, texto "Adding...", check icon a Psychology of waiting: sin feedback, 300ms se sienten eternos. Con feedback tiempo se siente instantáneo. Toast notifications: confirmación visual no-i

(auto-dismiss en 3s).

TAREA: Implementa estados de carga en botón "Add to Cart" con feedback visu

IMPLEMENTACIÓN:

- Archivo: src/features/product-catalog/components/AddToCartButton.tsx (cre
- Props: onAddToCart: () => Promise<void>, productName: string

BUTTON STATE PATTERN:

```
const [state, setState] = useState<'idle' | 'loading' | 'success'>('idle')

const handleClick = async () => {
  setState('loading') // Instant feedback

  try {
    await onAddToCart() // Simulate delay: await new Promise(r => setTimeo
    setState('success')
    // Reset after showing success
    setTimeout(() => setState('idle'), 2000)
  } catch {
    setState('idle')
  }
}
```

VISUAL STATES:

- idle: "Add to Cart" + cart icon
- loading: "Adding..." + spinner (button disabled)
- success: "Added!" + check icon (2s, luego vuelve a idle)

VERIFICAR:

1. Click en botón → cambio inmediato a "Adding..." con spinner
2. Simular delay 800ms con setTimeout
3. Success state muestra check icon
4. Después de 2s vuelve a estado idle
5. Durante loading, botón disabled previene double-click

VALIDACIÓN: Click debe mostrar feedback < 100ms, usuario VE que algo pasa i

Aprende: Estados de carga con feedback

instantáneo mejoran perceived performance más que optimizar velocidad real

Ejercicio 2: Optimistic UI con Rollback

Prompt:

Actúa como un frontend developer implementando optimistic UI pattern con ro

CONTEXT0: Optimistic UI = actualizar interfaz ANTES de confirmar con servid tradicional: Click → spinner → esperar 500ms → actualizar (usuario espera). optimista: Click → actualizar instantáneo → guardar background → rollback s (usuario NO espera). Rollback: revertir cambio si operación falla (crítico Error handling: mostrar toast + restaurar estado previo. Use cases ideales: increment/decrement, toggle settings (fácil revertir). Evitar para: payment deletion (difícil revertir). Simular API: setTimeout con random failure (20 para testing).

TAREA: Implementa optimistic UI para botón "Like Product" con rollback en e

IMPLEMENTACIÓN:

- Archivo: src/features/product-catalog/components/LikeButton.tsx (crear nu
- Props: productId: string, initialLiked: boolean

OPTIMISTIC PATTERN:

```
const [liked, setLiked] = useState(initialLiked)
const [likeCount, setLikeCount] = useState(0)

const handleToggleLike = async () => {
  // 1. Guardar estado previo para rollback
  const previousLiked = liked
  const previousCount = likeCount

  // 2. Actualizar UI INMEDIATAMENTE (optimistic)
  setLiked(!liked)
  setLikeCount(prev => liked ? prev - 1 : prev + 1)

  try {
    // 3. Simular API call (puede fallar 20% del tiempo)
    await new Promise((resolve, reject) => {
      setTimeout(() => {
        Math.random() > 0.2 ? resolve(true) : reject('Network error')
      }, 800)
    })
  } catch (error) {
    // 4. ROLLBACK si falla (restaurar estado previo)
```

```
    setLiked(previousLiked)
    setLikeCount(previousCount)
    // 5. Mostrar error al usuario
    alert('Failed to update like. Please try again.')
  }
}
```

VISUAL FEEDBACK:

- Liked: ❤️ rojo + count
- Not liked: 🤍 gris + count
- Instant toggle al hacer click (NO esperar)
- Si error: vuelve a estado previo + mensaje error

TESTING ROLLBACK:

1. Click en like → cambio instantáneo a ❤️ rojo
2. Si API falla (20% random) → vuelve a 🤍 + alert error
3. Click varias veces rápido → UI siempre responsive (optimistic)
4. Verificar count sube/baja correctamente
5. Verificar rollback restaura estado exacto previo

VALIDACIÓN: Like toggle debe ser instantáneo, rollback debe funcionar en ca

Aprende: Optimistic UI con rollback hace apps

10x más rápidas perceptualmente sin sacrificar confiabilidad

Puntos Clave

1. **Percepción > Realidad:** Cómo se siente importa más que la velocidad real
2. **< 100ms = Instantáneo:** No necesita feedback
3. **< 1s = Responsivo:** Indicador de carga simple
4. **> 3s = Lento:** Usuario probablemente abandonará
5. **Pantallas Esqueleto:** Mostrar estructura inmediatamente
6. **UI Optimista:** Actualizar instantáneamente, sincronizar en segundo plano
7. **Carga Progresiva:** Crítico primero, resto después
8. **Precarga:** Cargar siguiente paso antes de que usuario pida
9. **Feedback Instantáneo:** Cada click obtiene respuesta inmediata
10. **Cachear Agresivamente:** Rápido = desde caché, no red

